KitaHack

2026

Name: Chubaasiny Eswararao

Team Name: Better Call Java!

Project Name: SafeGo – Biometrics Based Personal Safety Application

Table of Contents

1.0 Introduction 3

2.0 Technical architecture 4

3.0 Implementation details 6

4.0 Challenges faced 8

5.0 Future roadmap 10

6.0 Conclusion 11

1.0 Introduction

In today's modern era, technology has become deeply embedded in daily life, offering not only convenience but also enhanced security through mobile applications. Despite these advancements, personal safety, especially for women and teenagers, remains a serious global concern. Reports from the World Health Organization (WHO, 2021) indicate that nearly one in three women has experienced violence, while statistics in Malaysia show thousands of missing children's cases in recent years (Bernama, 2024). Although many victims are found, even a small percentage of unresolved cases represents a critical and heartbreaking issue. The increasing risks have led individuals to avoid public transport and alter their routines, which affects public trust and overall societal confidence in safety systems.

To address these challenges, this project introduces SafeGo, a biometric based safety alert mobile application designed to enhance personal protection during travel. SafeGo provides real-time journey tracking, biometric interval check-ins, secure authentication, live location sharing, SOS emergency alerts and AI-assisted guidance during emergencies. By combining proactive monitoring with smart authentication and instant support, SafeGo empowers users, particularly high-risk individuals with greater confidence, control and peace of mind while commuting in public environments.

2.0 Technical architecture

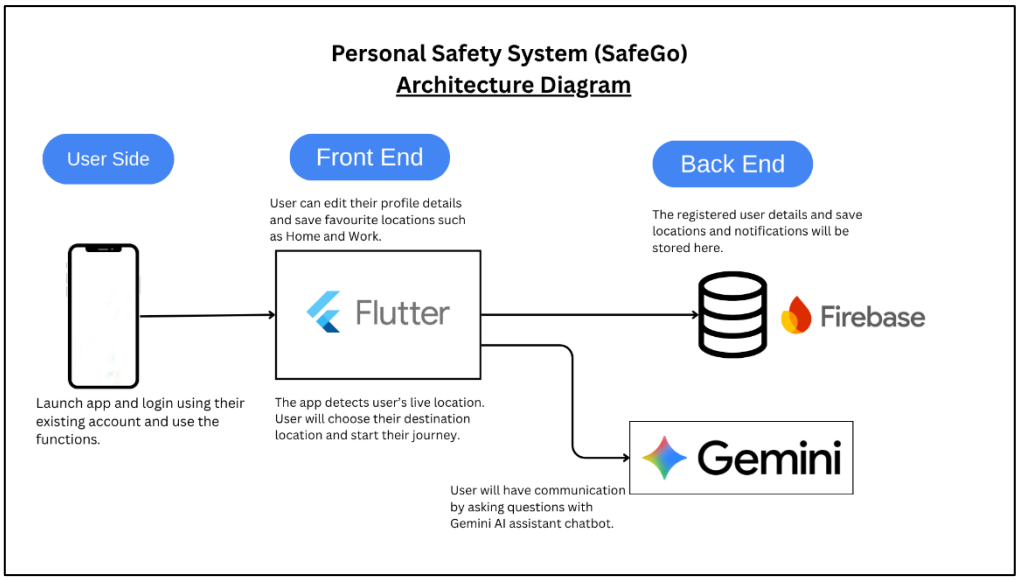

Figure 2.0: Architecture Diagram

The SafeGo application is designed using a modern mobile-cloud architecture that integrates secure authentication, real-time tracking, biometric verification, push notification services, and AI-powered assistance. The system follows a client-server architecture where the Android mobile application acts as the frontend client, while cloud-based services handle data storage, authentication, routing, and AI processing.

At the core of the system is the Android mobile application, which provides the user interface and controls all user interactions, including registration, biometric verification, journey tracking, SOS triggering and chatbot communication. The frontend communicates securely with Google Firebase, which functions as the backend infrastructure. Firebase Authentication manages user sign-up and login processes with email and biometric-based multi-factor authentication. The Firebase Firestore serves as the real-time database, storing user profiles, emergency contacts, journey data, live location updates and notification records.

For real-time journey tracking and route calculation, the system integrates the Open-Source Routing Machine (OSRM) Maps API. This API processes the user's starting point and destination to compute the optimal route, estimate travel duration and support interval-based biometric check-ins. The user's live GPS coordinates are continuously updated and stored in the database during the journey, enabling emergency contacts to monitor movement when permission is granted.

Push notification functionality is handled using Firebase Cloud Messaging (FCM). When an event occurs, such as journey start, missed check-in, safe arrival, journey cancellation or manual SOS activation, the system automatically triggers push notifications to the registered emergency contacts using device tokens stored in the database.

Additionally, SafeGo integrates the Gemini 2.5 Flash API to power its AI chatbot feature. The chatbot processes user queries related to emergency situations and returns contextual guidance in real time. The mobile application sends user prompts securely to the API endpoint and the AI-generated responses are displayed within the chatbot interface.

Overall, the SafeGo technical architecture ensures secure communication between components, real-time data synchronization, controlled access permissions and scalable cloud-based infrastructure. The integration of authentication services, routing APIs, cloud messaging and AI support creates a robust ecosystem designed to enhance personal safety during travel.

3.0 Implementation details

Table 0.0: Hardware Requirements

Hardware	Specification
Laptop	8 GB RAM, Intel Core i5 processor, 256 GB SSD, Windows 10 (64-bit)
Android Smartphone	Minimum of Android 8.0 or above
iphone	Minimum iOS 13 or above

Table 0.1: Software Requirements

Software	Specification
Visual Studio Code	Version 1.102.3 for code editing for flutter development.
Flutter	Version 3.32.1 for cross-platform app development framework.
OSRM Maps API	Version 6.0.0 for real-time GPS tracking and mapping.
Firebase SDK	Version 12.1.0 for authentication, database, and cloud messaging.
Google Gemini API	Version gemini-2.5-pro-002 for AI-powered features such as text analysis or chatbot responses.

The implementation of SafeGo was carried out using a combination of development tools, cloud services, APIs and structured database design to transform the proposed architecture into a functional mobile application. The system was developed using Visual Studio Code (Version 1.102.3) as the primary code editor with Flutter (Version 3.32.1) as the cross-platform development framework. Android Studio was installed to support emulator testing, debugging, and SDK management. Development was conducted on a laptop equipped with 8 GB RAM, Intel Core i5 processor, 256 GB SSD, running Windows 10 (64-bit). The application was frequently tested on a physical Android device (Samsung A13, Android 8.0 or above) to ensure real-world GPS accuracy, biometric functionality and notification delivery performance.

The backend infrastructure was implemented using Firebase SDK (Version 12.1.0). Firebase Authentication was configured to manage secure user registration and login using email credentials and biometric-based verification. Firebase Firestore served as the real-time database, supporting Create, Read, Update and Delete (CRUD) operations for user profiles, emergency contacts, saved locations, live journey tracking, and notification records. The database was structured with relational logic, where the main "Users" collection connects to sub-collections such as Emergency Contacts, Live Locations, Notifications, FCM Tokens and Connected Devices. Access control toggles were implemented to allow users to grant permissions for features such as live location sharing, safe arrival alerts and SOS-based tracking.

For routing and journey calculation, the Open-Source Routing Machine (OSRM) API was integrated to compute optimal travel routes and estimate journey duration. The system retrieves GPS coordinates, validates destinations and schedules interval-based biometric check-ins based on travel time. Firebase Cloud Messaging (FCM) was implemented to handle push notifications, automatically alerting emergency contacts during events such as journey start, missed check-ins, manual SOS activation, arrival confirmation, or journey cancellation.

To support intelligent emergency assistance, the Gemini 2.5 Flash API was integrated via REST API calls to power the SafeGo chatbot. User queries are securely processed, and contextual guidance is returned in real time within the application interface.

Comprehensive system testing was conducted through predefined test cases covering registration, authentication, biometric updates, real-time tracking, SOS triggering, chatbot responses, and notification delivery. All test cases achieved the expected outcomes, confirming system reliability, functional accuracy, and alignment with project objectives. The implementation phase successfully transformed the system design into a secure, scalable, and user-centered safety application.

4.0 Challenges faced

During the development of SafeGo, several technical and design challenges were encountered that influenced how the system was implemented:

a. **Integration of Maps for Route Tracking:** Implementing journey tracking using OSRM was one of the major challenges. The system had to calculate routes, estimate distance, and determine the expected travel time based on user input. Ensuring smooth rendering of routes, handling API responses, and updating real-time location continuously required careful data synchronization and optimization. This was addressed by optimizing update intervals, validating API responses before rendering, and reducing unnecessary UI refreshes to ensure smooth tracking.

b. **Determining Interval Check-In Logic:** Designing the biometric interval check-in system was challenging. The check-in interval needed to be aligned with the estimated journey time while ensuring that the verification prompts did not appear too frequently or too rarely. Hence, balancing usability, safety and system performance required multiple adjustments and testing. The solution implemented dynamic interval calculation based on journey duration, providing a balance between safety and convenience.

c. **Device-to-Device Communication Using FCM:** Creating a reliable communication link between the user's device and the emergency contact's device was another challenge. Connecting two separate devices through Firebase Cloud Messaging (FCM) required handling token registration, mapping users to their emergency contacts and ensuring that notifications were delivered instantly and consistently. This was particularly difficult when simulating real-world scenarios, such as when the SOS alert must immediately reach the emergency contact. This was solved by improving token refresh handling, verifying user-contact mappings, and adding pre-send validation to ensure reliable alert delivery.

d. **Maintaining Real-Time Accuracy and Smooth Performance:** Features such as live location sharing, map updates and biometric prompts run continuously. Hence, ensuring that the app remained responsive even during multiple asynchronous operations were occurring in the background required careful design and testing. Performance was improved by isolating background services from the UI, optimizing async handling, and minimizing unnecessary state refreshes.

5.0 Future roadmap

Based on the literature review and usability survey, there are some future enhancements can be implemented to this project.

Firstly, an automatic calling to emergency contacts, as well as the ability to send live location alerts directly to the nearest police station along with a short-coded voice message that an attacker cannot be easily understand. Secondly, the system can be implemented with fake calls that helps users to escape from uncomfortable situations. In addition, instant loud alarms such as a police siren can be implemented, allowing the user to draw attention and potentially scare the attacker away from the scene. Thirdly, an automatic audio or video recording to capture evidence during danger will be useful in investigations. Fourthly, enabling live location sharing even without internet, identifying and marking dangerous spots on the map and ensuring the system minimizes battery drain would enhance user safety and app reliability.

Next, features such as accident detection, face mood detection, hidden camera detection and capturing the attacker's face if the phone is snatched could further enhance user protection. Lastly, a community-based safety feature which is similar to WhatsApp Communities would be useful. The individuals can report if they faced any suspicious behaviours such as unsafe drivers' detail. As a result, others will stay informed and easily avoid any potential problems.

Finally, accessibility improvements will ensure SafeGo is inclusive for users with disabilities. Features such as voice commands, screen reader compatibility, larger buttons, haptic feedback and simplified navigation will allow visually, hearing or motor-impaired users to use the app safely and independently.

Overall, these improvements will take SafeGo to the next level, creating more comprehensive, reliable, inclusive and community-driven safety ecosystem.

6.0 Conclusion

In conclusion, SafeGo was successfully designed and implemented as a biometric-based personal safety mobile application that addresses real-world security concerns faced by individuals during travel. With increasing cases of violence, missing persons and safety-related incidents, there is a clear need for proactive and technology-driven safety solutions. SafeGo responds to this need by integrating real-time journey tracking, biometric interval verification, secure authentication, SOS emergency alerts, push notification services and AI-powered guidance within a unified mobile platform.

The technical architecture, built on a modern mobile-cloud infrastructure using Flutter, Firebase, OSRM API, Firebase Cloud Messaging and Gemini AI integration, ensures secure communication, real-time synchronization and scalable performance. Through systematic implementation, structured database design and comprehensive system testing, the proposed architecture was successfully transformed into a functional and reliable application.

Despite challenges related to real-time tracking, interval logic optimization, device-to-device communication and asynchronous performance management, effective debugging and architectural refinements strengthened the system's stability and responsiveness. These challenges contributed to improving SafeGo's reliability, accuracy and user experience.

With further enhancements outlined in the future roadmap, including advanced emergency automation, community-based safety networks, offline capabilities, AI-based detection features and accessibility improvements, SafeGo has a strong potential to evolve into a comprehensive and inclusive personal safety ecosystem.

Overall, SafeGo demonstrates how the integration of biometric security, cloud infrastructure, intelligent routing and AI assistance can create a practical, scalable and socially impactful solution that empowers users with greater confidence, safety and peace of mind in public environments.